

# LAMP: A Selective-ACK Transport Protocol for Reliable Bulk Data Transmission over LoRa Networks

Alessandro Monticelli

**Abstract**—This paper presents Lightweight Addressed Messaging Protocol (LAMP), a lightweight transport architecture that enables the continuous transfer of large data blocks over point-to-point Long-Range (LoRa<sup>®</sup>) networks. To overcome the prohibitive Time-on-Air overhead and duty-cycle saturation typical of existing Stop-and-Wait solutions, the proposed architecture segments payloads into up to 255 chunks per transmission window. Each chunk is protected by a 9-byte Logical Header with compact 8-bit node addressing (supporting downstream IPv6 link-local prefix mapping) and a 2-byte Cyclic Redundancy Check (CRC)-16 checksum suffix. The protocol provides four distinct operational modes combining connection state (Connectionless / Stateful) and reliability (Best-effort / SACK-ARQ feedback). By requiring receiver feedback only at sequence boundaries rather than per-packet, the Selective Acknowledgment (SACK) mechanism significantly reduces acknowledgment overhead without incurring the constant over-the-air encoding overhead of pure Forward Error Correction (FEC).

**Index Terms**—Long Range (LoRa), SACK, packet fragmentation, transport protocol, IoT, embedded systems, 3-Way Handshake, point-to-point.

## I. INTRODUCTION

**L**OW-POWER, low-cost, and long-range wireless communication technologies are becoming increasingly important in the context of the Internet of Things (IoT). Applications such as environmental monitoring, precision agriculture, and autonomous Unmanned Aerial Vehicles (UAVs) require reliable data transmission over extended distances under strict power and hardware constraints [1].

Several communication standards, including ZigBee, Sigfox, and Long Range (LoRa) have been adopted to address these needs, and LoRa, in particular, has emerged as the leading choice for low-power and long-range deployments thanks to its robust modulation technique and open network architecture [2]. However, conventional solutions based on LoRa (particularly those following the LoRaWAN<sup>®</sup> specifications) are optimized for small, non-frequent payloads [3].

This limitation poses acute restrictions on data-intensive IoT applications which demand the continuous transfer of large data blocks [4], [5], such as Firmware Update Over The Air (FUOTA), Simultaneous Localization and Mapping (SLAM) map transfers in autonomous robotics, or bulk sensor log collection. Standard Long Range Wide Area Network (LoRaWAN) links require substantial transmission overhead, restrictive duty-cycle limitations [6], and lack optimized mechanisms for handling large, multi-packet payloads in point-to-point configurations.

This paper presents the design and implementation of LAMP, a lightweight transport protocol for point-to-point and local multi-node LoRa networks. LAMP supports payloads up to 62.2 KB on 255-byte-First-In, First-Out (FIFO) transceivers (such as the Semtech<sup>®</sup> SX1262), operating independently of higher-layer middleware such as LoRaWAN.

The main contributions of this work are:

- **Bitmap-Based SACK Scheme:** Unlike existing Low-Power Wide-Area Network (LPWAN) fragmentation solutions that rely on Stop-and-Wait (S&W) Automatic Repeat Request (ARQ), which requires one acknowledgment per packet and incurs substantial latency under duty-cycle constraints, the proposed protocol uses a compact binary bitmap carried in a single feedback frame. The receiver reports the delivery status of up to 255 chunks per round-trip, reducing retransmission overhead and over-the-air footprint.
- **Lightweight Packet Segmentation and Abstraction:** The protocol introduces a 9-byte logical header (with 8-bit local node addressing) and a custom CRC integrity check on partial payloads, enabling out-of-order chunk reassembly on resource-constrained microcontrollers.
- **Modular C++17 Implementation and Test-Driven Development (TDD) Validation:** The core state machine is completely decoupled from the physical radio driver through a Hardware Abstraction Layer (HAL). This design enables both native host-based unit testing (using the Unity framework) and physical deployment on Semtech SX126x transceivers.

The remainder of this paper is organized as follows. Section II surveys related work and defines the design space. Section III details the system architecture, packet structure, and formal protocol algorithms. Section IV outlines preliminary prototyping findings. Section V presents the hardware-level C++17 implementation. Section VI evaluates performance through extensive ns-3 simulations and planned field validation. Section VII concludes the paper and presents the architectural roadmap.

## II. STATE OF THE ART

Reliable packet fragmentation and reassembly over LPWANs has been the subject of active research, driven by the growing demand for data-intensive IoT applications.

A prominent industrial standard is Static Context Header Compression (SCHC) [7], [8], which introduces a generic

framework for header compression and fragmentation over LPWANs. In particular, RFC 9011 defines SCHC over LoRaWAN profiles supporting No-ACK, ACK-Always, and ACK-on-Error modes. While SCHC excels in IP-connected topologies with pre-provisioned rule contexts, LAMP is designed as an autonomous, zero-configuration transport layer operating directly over raw LoRa PHY. Similarly, the LoRa Alliance standardized Fragmented Data Block Transport (**TS004!**) [9] to support FUOTA using open-loop FEC redundancy codes to reconstruct blocks without feedback. However, open-loop FEC introduces constant over-the-air encoding overhead regardless of channel conditions and requires compute-intensive decoding on embedded microcontrollers.

For custom peer-to-peer LoRa networks, early efforts focused on overcoming the excessive airtime latencies and capacity constraints of traditional Stop-and-Wait (S&W) ARQ schemes. Empirical link evaluations by Cattani et al. [10] and network scalability analyses by Bor et al. [11] demonstrated that high packet-loss rates and strict Time-on-Air limits make per-packet Stop-and-Wait acknowledgments unviable for bulk data transfers [4]. Chen et al. [12] proposed the Multi-Packet LoRa (MPLR) protocol, introducing the concept of replacing Stop-and-Wait ARQ with a windowed batch transmission scheme acknowledged by a bit-vector acknowledgment packet (BVACK). Subsequent specialized studies explored multi-hop relay networks for visual IoT coverage [13], comprehensive mesh survey frameworks [14], and Time Division Multiplexing (TDM) scheduled channels [15]. As a pure transport-layer protocol, LAMP is topology-agnostic and can operate transparently on top of multi-hop mesh routing protocols to extend reliable bulk delivery across multi-hop topologies.

Building on the bit-vector SACK concept, LAMP extends and generalizes this approach with several key protocol-level optimizations:

- **Header and Protocol Overhead Optimization:** MPLR relies on a 16-byte header per fragment (carrying 4-byte Destination EUI, 4-byte Source EUI, Service Number, Sequence Number, Flags, Payload Size, Batch Size, and Checksum). In contrast, LAMP utilizes a 9-byte Logical Header (1-byte `srcAddr`, 1-byte `dstAddr`, 2-byte `messageId`, 1-byte `totalChunks`, 1-byte `chunkIndex`, 1-byte `payloadSize`, 1-byte `flags`, and 1-byte `protocolVersion`). Including the 2-byte CRC-16 checksum suffix, LAMP requires a total over-the-air overhead of 11 bytes per frame compared to MPLR's 16 bytes, yielding a 5-byte reduction in total over-the-air overhead per fragment.
- **Target Scope:** MPLR was engineered specifically as an application-level image transmission protocol for agricultural uplink nodes to a central gateway. LAMP is designed as a domain-agnostic, general-purpose transport layer capable of segmenting any arbitrary data payload up to 62.2 KB on standard LoRa transceivers (ranging from high-frequency sensor telemetry arrays and robotic SLAM keyframes to firmware binaries and multiagent state matrices).
- **Flexible Operational Modes:** MPLR operates under a single stateful batch delivery model. In contrast, LAMP

provides four operational modes spanning *Unreliable Connectionless* (best-effort datagrams), *Unreliable Connected* (session-tracked streaming), *Reliable Connectionless* (stateless SACK-ARQ), and *Reliable Connected* (stateful negotiated streams).

- **Dynamic Parameter Negotiation:** MPLR utilizes a fixed batch size and requires a 4-way handshake tear-down (FIN/ACK). LAMP implements a lightweight 3-Way Handshake (3WHS) embedded with dynamic parameter negotiation (`SynMetadata` and `SynAckMetadata` negotiating chunk size, sliding window, and inactivity timeout) and replaces explicit tear-down overhead with automatic idle session pruning.

To clearly position the proposed protocol within the current technological landscape and define the design space in which LAMP operates, Table I provides an architectural comparison with other prominent fragmentation and transport solutions over LoRa, including Stop-and-Wait ARQ, SCHC [7], [8], LoRaWAN Fragmented Data Block Transport (**TS004!**) [9], and MPLR [12].

Rather than claiming overall superiority over existing industrial standards, LAMP addresses a distinct design space in raw LoRa communications: providing a *lightweight, autonomous, and configurable transport layer for raw LoRa point-to-point and local multi-node links*. Each protocol in Table I represents a distinct set of engineering trade-offs tailored to specific deployment constraints:

- **SCHC [7], [8]:** Represents the standard for IP-connected LPWANs, achieving header compression down to 1–2 bytes for predictable IPv6/UDP flows. However, it relies on pre-shared static context rules, making it less suitable for standalone, ad-hoc, raw-LoRa peer-to-peer or multi-node links without gateway context managers.
- **LoRaWAN Fragmented Transport (TS004!) [9]:** Optimized for unidirectional downlink firmware updates (FUOTA) to thousands of nodes simultaneously using Forward Error Correction (FEC) without requiring a feedback channel. However, open-loop FEC introduces constant over-the-air encoding overhead regardless of channel quality and imposes significant computational decoding overhead on low-cost microcontrollers.
- **MPLR [12]:** Engineered specifically for high-volume agricultural image uplink transfers (e.g., 9 KB to multi-megabyte images) using 40-packet batching with bit-vector acknowledgments over a 16-bit sequence space (up to 65,536 distinct message IDs). However, it utilizes a 16-byte fixed header (8-byte EUI addressing), operates under a single stateful batch mode, and requires explicit 4-way handshake teardown (FIN/ACK).
- **Stop-and-Wait ARQ:** Offers minimal header size (2–4 bytes) and trivial stateless implementation for isolated single-packet telemetry. However, it incurs prohibitive airtime latencies and strict ETSI duty-cycle saturation when applied to multi-fragment payloads due to per-packet ACK waiting delays.
- **LAMP (Proposed Protocol):** Fills the design space of standalone raw-LoRa links by providing four con-

figurable operational modes (Reliable/Unreliable  $\times$  Connected/Connectionless), 8-bit local unicast/broadcast addressing, dynamic 3WHs buffer negotiation, and transport-layer purity (decoupled from opt-in HAL channel access helpers like CAD/CSMA-CA). *Protocol Limitations & Trade-offs*: To optimize header efficiency, LAMP allocates 1 byte for fragment indexing (`totalChunks` and `chunkIndex`), bounding a single SACK bitmap session to 255 chunks (62.2 KB total payload size, requiring sequential sessions for larger files). Furthermore, carrying 8-bit node addresses and sequence metadata requires a 9-byte Logical Header (+2-byte CRC-16 suffix, totaling 11 bytes overhead per frame), and session tracking requires microcontroller RAM allocation for SACK bitmaps (dynamically sized up to 32 bytes for 255 chunks,  $\lceil \text{totalChunks}/8 \rceil$ ) and millisecond timer primitives, unlike stateless Stop-and-Wait ARQ.

### III. PROPOSED SYSTEM ARCHITECTURE

LAMP is a transport-layer protocol running directly over the LoRa physical layer. It segments arbitrarily large payloads (within the described constraints) at the transmitter and reconstructs them at the receiver while minimizing Logical Header and Over-the-Air (OTA) overhead.

#### A. Protocol Specification and Design Iterations

We describe the design of LAMP by contrasting its initial stateless baseline (LAMPv1) with the current addressed architecture (LAMPv2).

1) *Stateless Header Specification*: The initial protocol version, LAMPv1, was designed for maximum header compression during periodic sensor telemetry transfers over point-to-point links. It utilized a compact 7-byte Logical Header containing only sequence tracking and payload bounds (`messageId` [2 B], `totalChunks` [1 B], `chunkIndex` [1 B], `payloadSize` [1 B], `flags` [1 B], and `protocolVersion` [1 B]).

While LAMPv1 achieved high spectral efficiency for isolated datagram bursts, deployment on embedded hardware revealed three key architectural limitations:

- *Speculative Buffer Allocation*: Receivers allocated reassembly memory speculatively upon receiving the first fragment of an unannounced stream. When receiving large payloads (e.g., image tiles or firmware blocks), memory-constrained microcontrollers risked heap exhaustion or buffer overflows.
- *Session Interference*: Lacking explicit source and destination identifiers, multiple unaddressed transmitters sharing the same RF channel risked interleaving fragments with identical sequence numbers, leading to corrupted reassembly buffers.
- *Static Transport Limits*: Fixed chunk sizes and static timeout thresholds prevented dynamic adaptation to link quality degradation or variable receiver memory limits.

2) *Addressed Logical Header Specification*: To resolve the limitations of LAMPv1, LAMPv2 introduces an addressed, stateful transport architecture. LAMPv2 expands the Logical Header to 9 bytes by embedding explicit 8-bit Source (`srcAddr`) and Destination (`dstAddr`) Node Identifiers, while incorporating an integrated 3WHs mechanism.

Although adding 2 bytes to the header increases over-the-air frame length by less than 0.8% relative to the maximum 255-byte frame (9-byte header + 244-byte payload + 2-byte CRC-16), this trade-off provides three main architectural benefits:

- 1) *Buffer Memory Safety*: The 3WHs exchange allows nodes to negotiate reassembly buffer limits (`SynMetadata`) prior to payload transmission, preventing allocation failures on memory-constrained receivers.
- 2) *Channel Session Isolation*: Explicit node addressing prevents session crosstalk and frame interleaving in multi-node wireless topologies.
- 3) *IPv6 Integration Readiness*: The 8-bit Node ID introduces a lightweight local addressing namespace that can be mapped directly to an IPv6 link-local prefix at an edge gateway, enabling transparent IP-layer integration without transmitting heavy 128-bit IPv6 headers over raw LoRa radio links.

#### B. Operational Modes and Sequence Management

To accommodate diverse IoT workload requirements, LAMP defines four distinct operational modes across two primary dimensions: *Reliability* (Unreliable Best-Effort vs. SACK-ARQ) and *Connection State* (Connectionless Handshake-Free vs. Connection-Oriented Stateful).

1) *Implicit Frame Boundary Derivation*: To maintain a compact 9-byte header while supporting robust out-of-order packet reassembly, LAMP avoids dedicated boundary control bits. Instead, Start-of-Message (SOM) and End-of-Message (EOM) boundaries are implicitly derived at the receiver directly from the 0-based fragment index (`chunkIndex`) and the total fragment count (`totalChunks`):

$$\text{SOM} \iff \text{chunkIndex} == 0 \quad (1)$$

$$\text{EOM} \iff \text{chunkIndex} == \text{totalChunks} - 1 \quad (2)$$

Explicitly carrying `totalChunks` in every fragment header allows the receiver to determine the exact message bounds and pre-allocate the required reassembly buffer and SACK bitmap upon receiving *any* fragment, even if fragments arrive out of order or if the initial chunk is lost.

2) *Node Addressing Scheme*: To support multi-node communication without incurring the prohibitive airtime overhead of full 128-bit IPv6 headers on LoRa PHY payloads, LAMP introduces a compact 8-bit Node Identifier space:

- **0x00 (ADDRESS\_UNASSIGNED)**: Reserved for legacy or unaddressed P2P datagrams (promiscuous mode).
- **0x01–0xFE**: 254 unique individual node addresses (Unicast).
- **0xFF (ADDRESS\_BROADCAST)**: Broadcast address processed by all active receiving nodes on the RF channel.

TABLE I  
ARCHITECTURAL COMPARISON AND TRADE-OFF MATRIX OF LoRa TRANSPORT SOLUTIONS

| Architectural Dimension      | Stop-and-Wait ARQ                                  | SCHC [7]                                         | LoRaWAN Fragmented [9]                           | MPLR [12]                                      | Proposed (LAMP)                                                |
|------------------------------|----------------------------------------------------|--------------------------------------------------|--------------------------------------------------|------------------------------------------------|----------------------------------------------------------------|
| Stack Dependency             | Raw LoRa PHY                                       | Generic Adaptation (often LoRaWAN/IP)            | LoRaWAN App Layer (TS004!)                       | Raw LoRa PHY                                   | Raw LoRa PHY (Standalone)                                      |
| ACK Mechanism                | Per-Packet ACK                                     | ACK-on-Error / ACK-Always                        | None (Open-Loop Redundancy)                      | Bit-Vector SACK                                | Bit-Vector SACK                                                |
| Header Overhead              | 2–4 Bytes                                          | 1–2 Bytes (Compressed RuleID/Residue)            | 2 Bytes                                          | 16 Bytes                                       | 9 Bytes (11B total w/ CRC)                                     |
| Max Frame Payload            | 240+ Bytes                                         | Variable                                         | Variable                                         | 239 Bytes                                      | 244 Bytes (255B PHY limit)                                     |
| Max Session Payload          | Unbounded (Sequential)                             | Variable (IPv6 Packet)                           | Unlimited (Block Session)                        | Multi-MB Image (65k Packets)                   | 62.2 KB (255 Chunks / Session)                                 |
| Operational Modes            | 1 (Reliable S&W)                                   | 3 (No-ACK, ACK-Always, ACK-on-Error)             | 1 (Open-Loop FEC)                                | 1 (Reliable Batch)                             | 4 Configurable Modes                                           |
| Node Addressing              | None / Application                                 | IPv6 128-bit (Compressed)                        | DevAddr (32-bit)                                 | EUI (64-bit)                                   | 8-bit Unicast + Broadcast                                      |
| Multi-Node Suitability       | Single P2P Link                                    | Star (Gateway-Centric)                           | Star Broadcast (Downlink)                        | P2P Uplink (Node-GW)                           | P2P & Multi-Node Ready                                         |
| Parameter Negotiation        | None (Static)                                      | Static (Pre-shared Rules)                        | Static / Out-of-Band                             | Static Batch / 4WS                             | Dynamic 3WS Metadata                                           |
| Key Strengths                | Minimal header size, trivial implementation        | Header compression (1–2 B) for IP traffic        | No feedback channel required (pure FEC)          | High-volume image uplink batching              | 4 flexible modes, dynamic 3WS buffer safety, raw LoRa autonomy |
| Key Limitations & Trade-offs | Prohibitive airtime latency, duty-cycle saturation | Requires shared rule context & IP infrastructure | High FEC compute cost, constant airtime overhead | 16B header overhead, single mode, 4WS teardown | 9B header (11B w/ CRC), 62.2 KB session limit, RAM/timer state |

TABLE II  
LAMP OPERATIONAL MODES ARCHITECTURE

| Operational Mode                      | Reliability & Connection State                   | Primary Target Use Case                                                                           |
|---------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Mode 1: Best-Effort Datagram</b>   | Unreliable, Connectionless (Stateless)           | High-frequency telemetry streams, periodic sensor data bursts.                                    |
| <b>Mode 2: Best-Effort Stream</b>     | Unreliable, Connection-Oriented (Stateful)       | Session-tracked real-time sensor streams without retransmission delays.                           |
| <b>Mode 3: SACK-Reliable Datagram</b> | Reliable (SACK), Connectionless (Handshake-Free) | Isolated high-priority sensor bursts requiring delivery confirmation without prior session setup. |
| <b>Mode 4: SACK-Reliable Stream</b>   | Reliable (SACK), Connection-Oriented (3WS)       | Critical large payload transfers (FUOTA binaries, camera images, SLAM maps).                      |

As detailed in Section III-A, this 8-bit Node ID maps directly to an IPv6 link-local suffix at edge gateways, enabling IP-layer integration without transmitting 128-bit addresses over the constrained radio link.

The 8-bit address space supports up to 254 unicast nodes per RF cluster, which is sufficient for the dense but bounded deployments targeted by this work (e.g., agricultural sensor arrays, UAV swarm sub-clusters). For applications requiring broader network reach, Section VII foresees integration of full IPv6 routing, where the 8-bit node identifier serves as a local cluster suffix rather than a global unique identifier. Note that the 254 addressable nodes operate independently of the receiver's concurrency limit ( $\text{MAX\_CONCURRENT\_SESSIONS} = 10$ ), which governs the maximum number of simultaneously active reassembly sessions.

The exact byte alignment and offsets of the 9-byte header packet structure are presented in Table III and Figure 1.

TABLE III  
LAMP OTA PACKET LAYOUT AND BYTE OFFSETS

| Offset   | Field Name      | Type      | Description                                                 |
|----------|-----------------|-----------|-------------------------------------------------------------|
| 0        | srcAddr         | uint8_t   | Source Node ID (0x01–0xFE)                                  |
| 1        | dstAddr         | uint8_t   | Destination Node ID (0x01–0xFE, 0xFF)                       |
| 2–3      | messageId       | uint16_t  | Message Sequence ID                                         |
| 4        | totalChunks     | uint8_t   | Total Count ( $1 \leq N \leq 255$ ; $N = 0$ invalid)        |
| 5        | chunkIndex      | uint8_t   | Index ( $0 \leq i < N$ )                                    |
| 6        | payloadSize     | uint8_t   | Chunk Payload ( $0 \leq L \leq 244$ B; $L = 0$ for control) |
| 7        | flags           | uint8_t   | Bitfield (ACK_REQ, CONN_REQ)                                |
| 8        | protocolVersion | uint8_t   | Protocol Version (0x02)                                     |
| 9–L+8    | payload[]       | uint8_t[] | Application Data (L bytes)                                  |
| L+9–L+10 | crc             | uint16_t  | CRC-16 Checksum                                             |

| Byte 0  | Byte 1  | Byte 2 – 3 | Byte 4      | Byte 5     | Byte 6      | Byte 7 | Byte 8          |
|---------|---------|------------|-------------|------------|-------------|--------|-----------------|
| srcAddr | dstAddr | messageId  | totalChunks | chunkIndex | payloadSize | flags  | protocolVersion |

Fig. 1. Byte layout of the compact 9-byte Logical Header incorporating Node Addressing.

The fields in the Logical Header are defined as follows:

- **srcAddr (1 byte):** Source node identifier (0x01–0xFE, or 0x00 for unassigned).
- **dstAddr (1 byte):** Destination node identifier (0x01–0xFE, or 0xFF for broadcast).
- **messageId (2 bytes):** Unique message sequence identifier. All fragments composing the same segmented message share this ID. Active and completed sessions are uniquely keyed by the 4-tuple (srcAddr, dstAddr, messageId, version).
- **totalChunks (1 byte):** The total count of chunks into which the original large payload is divided ( $1 \leq \text{totalChunks} \leq 255$ ).
- **chunkIndex (1 byte):** The 0-based index of this specific fragment ( $0 \leq \text{chunkIndex} < \text{totalChunks}$ ).
- **payloadSize (1 byte):** Payload data byte count ( $0 \leq L \leq 244$  bytes;  $L = 0$  for pure control frames). On standard SX1262 transceivers (256-byte FIFO, 255-byte maximum PHY frame), subtracting the 9-byte header and 2-byte CRC-16 yields  $S_{\max} = 244$  bytes per chunk, resulting in a single-session capacity of  $255 \times 244 = 62,220$  bytes ( $\approx 62.2$  KB).
- **flags (1 byte):** A control bitfield used to manage transmission and connection states:
  - FLAG\_ACK\_REQ (0x04): Selective Acknowledgment requested from receiver.
  - FLAG\_ACK (0x08): SACK feedback packet containing binary bitmap of received chunks.
  - FLAG\_CONN\_REQ (0x10): Connection Request control frame.
  - FLAG\_CONN\_ACK (0x20): Connection Acknowledged control frame.
  - FLAG\_CONN\_NACK (0x40): Connection Refused control frame.
- **protocolVersion (1 byte):** Identifies protocol version (currently 2) to ensure backward compatibility.

In LAMPv2, broadcast addressing (0xFF) is strictly restricted to unreliable Best-Effort Datagrams (Mode 1). To avoid acknowledgment implosion and channel collapse, control frames (FLAG\_ACK\_REQ, SYN, SYN-ACK, CONN\_NACK) are prohibited with dstAddr = 0xFF.

After the payload, a 16-bit CRC-16 (Modbus CRC-16, polynomial 0xA001, initialized to 0xFFFF) is appended. The checksum is computed over the concatenation of the 9-byte Logical Header and exactly payloadSize application-layer bytes; any trailing buffer bytes beyond payloadSize are excluded, so that partial final chunks are validated identically to full-size chunks and inter-implementation interoperability is preserved.

Table IV summarises all defined flag combinations and the behaviour of the packet parser upon receipt of an undefined value.

TABLE IV  
LAMP FLAG FIELD: VALID COMBINATIONS AND PARSER BEHAVIOUR

| Value | Semantic              | Notes                               |
|-------|-----------------------|-------------------------------------|
| 0x00  | Data chunk            | Unreliable/reliable chunk, no ACK   |
| 0x04  | Final chunk (ACK_REQ) | Receiver replies with SACK          |
| 0x08  | SACK feedback         | Payload = received-chunk bitmap     |
| 0x10  | SYN                   | 3WHS Step 1 (SynMetadata)           |
| 0x30  | SYN-ACK               | 3WHS Step 2 (SynAckMetadata)        |
| 0x20  | ACK                   | 3WHS Step 3, no payload ( $L = 0$ ) |
| 0x40  | CONN-NACK             | Payload = rejection reason code     |
| other | Undefined             | Silently discarded                  |

### C. Selective Acknowledgment and Retransmission

In reliable mode, the protocol implements a SACK mechanism [16] to recover lost fragments with minimal additional airtime.

1) *Retransmission Protocol Interaction:* When a transmission is executed in reliable mode:

- 1) The transmitter segments the payload, serializes the chunks with srcAddr and dstAddr, and streams them sequentially. The final chunk index ( $\text{Index} = \text{totalChunks} - 1$ ) is marked with the FLAG\_ACK\_REQ flag.
- 2) Upon receipt of the chunk requesting acknowledgment, the receiver evaluates its reassembly session. It identifies which fragments are still missing by checking its internal bitmap of received chunks.
- 3) The receiver responds by transmitting a SACK packet (FLAG\_ACK) addressed back to the transmitter's srcAddr. The SACK carries a payload of  $\lceil N/8 \rceil$  bytes containing a binary bitmap: chunk index  $i$  ( $0 \leq i < N$ ) maps to bit position  $(i \bmod 8)$  of byte  $\lfloor i/8 \rfloor$  (LSB-first ordering), where a 1-bit indicates successful reception and a 0-bit indicates a missing fragment.
- 4) The transmitter parses the received SACK bitmap. If some chunks are missing, it selectively retransmits only those lost chunks. The last retransmitted chunk has the FLAG\_ACK\_REQ flag set again to query the receiver.
- 5) This selective retry loop continues until the full message is reconstructed at the receiver, or the transmitter reaches the maximum retry count.

2) *Half-Duplex Hardware Switching Guard:* Because LoRa transceivers are half-duplex, a hardware transition period is required for the transmitter to switch from transmit (TX) mode to receive (RX) mode after sending a packet. While physical SX1262 transceiver state transitions take  $< 100 \mu\text{s}$ , the  $\delta = 25$  ms guard delay accounts for physical transceiver switching alongside software-level Real-Time Operating System (RTOS) task scheduling jitter, Serial Peripheral Interface (SPI) bus transaction queuing, and microcontroller wake-up latencies on embedded targets.

It is important to emphasize that all protocol timing parameters, guard delays (e.g.,

SACK\_PREAMBLE\_GUARD\_DELAY\_MS), and retry limits are fully decoupled into a centralized configuration namespace (LoRaMultiPacketConfig) and can be customized by integrators for any MCU/radio target. The default parameters presented in this study were specifically tuned around the Heltec Wi-Fi LoRa 32 V3 (ESP32-S3) hardware platform.

3) *Timeout and Fallback Recovery*: If the transmitter does not receive a SACK packet within the dynamic timeout window, it assumes the query or response was lost and retransmits the last transmitted chunk with the FLAG\_ACK\_REQ flag. The transmitter permits up to  $R_{\max} = 5$  consecutive timeout events before aborting the session.

The timeout is computed adaptively to account for forward data frame airtime, receiver turnaround, and reverse SACK airtime:

$$T_{\text{timeout}} = \max\left(T_{\min}, \alpha \cdot \left(\hat{T}_{\text{OA,DATA}} + \hat{T}_{\text{OA,SACK}}\right) + \delta\right) \quad (3)$$

where  $T_{\min} = 1000$  ms is the minimum guard,  $\alpha = 1.5$  is a safety factor,  $\hat{T}_{\text{OA,DATA}}$  and  $\hat{T}_{\text{OA,SACK}}$  are the Times-on-Air for the data frame and reverse SACK frame queried from the radio driver [17], and  $\delta = 25$  ms is the half-duplex switching and scheduling guard delay. Table V reports representative timeout values across spreading factors.

4) *Idealized Analytical Burst Reliability Baseline*: Under an independent frame loss probability  $p$  and assuming error-free control feedback, the probability that an individual chunk fails after  $R_{\max}$  repair rounds is  $p^{R_{\max}+1}$ . For an  $N$ -chunk payload under this idealized i.i.d. baseline, the overall burst delivery success probability  $P_{\text{success}}$  is modeled as:

$$P_{\text{success}} = (1 - p^{R_{\max}+1})^N, \quad P_{\text{fail}} = 1 - P_{\text{success}} \quad (4)$$

For a lossy channel with a 20% frame error rate ( $p = 0.20$ ) and  $R_{\max} = 5$  (matching LoRaMultiPacketConfig::MAX\_RETRIES),  $P_{\text{success}} = (1 - 0.20^6)^{20} \approx 99.87\%$  for a 20-chunk transfer. In physical wireless environments with correlated shadowing and reverse-link fading, overall completion rates are evaluated empirically via the stochastic ns-3 campaign in Section VI.

TABLE V  
ADAPTIVE SACK TIMEOUT VALUES (BW 500 KHz, CR 4/5, 255-BYTE MAX FRAME)

| SF | $\hat{T}_{\text{DATA}}$ (ms) | $\alpha \sum \hat{T}_{\text{OA}} + \delta$ (ms) | $T_{\text{timeout}}$ (ms) |
|----|------------------------------|-------------------------------------------------|---------------------------|
| 7  | 99                           | 188                                             | 1000                      |
| 8  | 174                          | 312                                             | 1000                      |
| 9  | 313                          | 542                                             | 1000                      |
| 10 | 564                          | 958                                             | 1000                      |
| 11 | 1025                         | 1720                                            | 1720                      |
| 12 | 1886                         | 3140                                            | 3140                      |

#### D. Memory Bounding and RAM Footprint Analysis

To guarantee memory safety on constrained microcontrollers, the dynamic memory allocated for an active reassembly session at the receiver is strictly bounded by:

$$M_{\text{session}} = S_{\text{struct}} + N_{\text{chunks}} \cdot S_{\text{max}} + \left\lceil \frac{N_{\text{chunks}}}{8} \right\rceil \quad (5)$$

where  $S_{\text{struct}} \approx 48$  bytes represents the session tracking structure metadata,  $S_{\text{max}} = 244$  bytes is the maximum chunk stride, and the final term represents the binary SACK bitmask.

For a representative 20-chunk payload (4.88 KB data size), a complete reassembly session requires  $\approx 4.93$  KB of RAM (totaling  $\approx 49.3$  KB across 10 concurrent 20-chunk sessions). On entry-level microcontrollers with limited SRAM ( $\leq 64$  KB), the concurrency pool ceiling (MAX\_CONCURRENT\_SESSIONS) is tuned to 1–2 sessions in LoRaMultiPacketConfig, while runtime heap protection (AvailMem()) prevents allocation failures by rejecting new requests with CONN\_NACK(BUSY) when memory is depleted.

#### E. Formal Protocol Algorithms

To ensure complete specification clarity and system-level reproducibility across different hardware platforms, Algorithm 1 and Algorithm 2 formally detail the transmitter-side segmentation pipeline and the receiver-side out-of-order reassembly engine, respectively.

#### F. Session Control and Handshake Protocol

To support stateful stream negotiation and protect resource-constrained receivers from buffer overflows, LAMPv2 introduces a 3WHS mechanism.

1) *Three-Way Handshake Procedure*: As illustrated in Figure 3(a), the connection lifecycle follows a three-step exchange:

- 1) **SYN**: The initiator sends a SYN control frame (FLAG\_CONN\_REQ) addressed to the target node's dstAddr, containing a 4-byte SynMetadata payload (requestedPayloadSize, windowSize, timeoutMs).
- 2) **SYN-ACK**: Upon receiving the SYN, the peer evaluates available memory and protocol version. If accepted, it responds with a SYN-ACK control frame (FLAG\_CONN\_REQ | FLAG\_CONN\_ACK) addressed back to the initiator. The receiver clamps acceptedPayloadSize to its maximum buffer capability.
- 3) **ACK**: The initiator parses the SYN-ACK, adopts the negotiated payload size, and transmits a final ACK frame (FLAG\_CONN\_ACK). Both nodes transition their internal state machine to ESTABLISHED.

#### G. Concurrency and Fault Handling

To ensure protocol stability in multi-node and lossy environments, LAMP incorporates the following fault-handling mechanisms:

- *Simultaneous Connection Resolution*: If two nodes transmit SYN frames to each other simultaneously, a deterministic tie-breaking rule resolves the conflict: the node with the lower numerical NodeAddress yields and transitions back to CLOSED, while the node with the higher NodeAddress proceeds with session setup.

---

**Algorithm 1** LAMP Payload Transmission and SACK-ARQ (Transmitter)

---

**Require:** Payload  $P$  (size  $L$ ),  $dstAddr$ , Max Chunk Size  $S$ , Mode  $M$   
**Ensure:** Delivery Status (OK or FAIL)

```

1:  $N \leftarrow \lceil L/S \rceil$ 
2: if  $N > 255$  then
3:   return ERR_PAYLOAD_TOO_LARGE
4: end if
5:  $msgId \leftarrow \text{GetNextMsgId}()$ 
6: for  $i \leftarrow 0$  to  $N - 1$  do
7:    $cLen \leftarrow (i == N - 1) ? (L - i \cdot S) : S$ 
8:    $flags \leftarrow (M \in \{\text{Mode 3}, \text{Mode 4}\} \text{ and } i == N - 1) ? \text{FLAG\_ACK\_REQ} : 0x00$ 
9:    $hdr \leftarrow \text{BuildHdr}(src, dst, msgId, N, i, cLen, flags)$ 
10:   $crc \leftarrow \text{CalcCRC16}(hdr, P[i \cdot S : i \cdot S + cLen])$ 
11:   $F_i \leftarrow \text{SerializeFrame}(hdr, P[i \cdot S : i \cdot S + cLen], crc)$ 
12:   $\text{Transmit}(F_i)$ 
13: end for
14: if  $M \in \{\text{Mode 1}, \text{Mode 2}\}$  then
15:   return OK ▷ Unreliable delivery complete
16: end if
17:  $retries \leftarrow 0$ ;  $lastIdx \leftarrow N - 1$ 
18: while  $retries \leq R_{\max}$  do
19:    $sack \leftarrow \text{WaitForSack}(T_{\text{timeout}})$ 
20:   if  $sack == \text{TIMEOUT}$  then
21:     if  $retries == R_{\max}$  then
22:       break
23:     end if
24:      $\text{Transmit}(F_{lastIdx})$ ;  $retries \leftarrow retries + 1$ 
25:   else if  $\text{IsAllAacked}(sack.bitmap)$  then
26:     return OK ▷ Full payload delivery confirmed
27:   else
28:     if  $retries == R_{\max}$  then
29:       break
30:     end if
31:      $missing \leftarrow \text{MissingChunks}(sack.bitmap)$ 
32:     for  $idx \in missing$  do
33:        $cLen \leftarrow (idx == N - 1) ? (L - idx \cdot S) : S$ 
34:        $flags \leftarrow (idx == \text{Last}(missing)) ? \text{FLAG\_ACK\_REQ} : 0x00$ 
35:        $hdr \leftarrow \text{BuildHdr}(src, dst, msgId, N, idx, cLen, flags)$ 
36:        $crc \leftarrow \text{CalcCRC16}(hdr, P[idx \cdot S : idx \cdot S + cLen])$ 
37:        $F_{idx} \leftarrow \text{SerializeFrame}(hdr, P[idx \cdot S : idx \cdot S + cLen], crc)$ 
38:        $\text{Transmit}(F_{idx})$ 
39:     end for
40:      $lastIdx \leftarrow \text{Last}(missing)$ ;  $retries \leftarrow retries + 1$ 
41:   end if
42: end while
43: return ERR_MAX_RETRIES_EXCEEDED

```

---

- *Receiver Busy Rejection:* If a receiver receives a SYN frame while its active reassembly session is already locked by another node, it responds with a CONN\_NACK frame (FLAG\_CONN\_NACK) specifying a BUSY status code, causing the requester to back off.
- *Source Address Filtering:* Initiators in waitForSynAck and waitForSack validate incoming control frames against the expected  $srcAddr$ , ignoring packets from unrelated third-party nodes.
- *Incomplete Handshake Timeout:* If a SYN-ACK or ACK of a 3WHS is lost in transit, the receiver in SYN\_RCVD or initiator in SYN\_SENT automatically aborts and resets to CLOSED after an inactivity timeout ( $timeoutMs$ ).

---

**Algorithm 2** Out-of-Order Reassembly and SACK Generation (Receiver)

---

**Require:** Received Frame  $F$ , Fixed Chunk Stride  $S_{\max}$   
**Ensure:** Reassembled Payload  $P$  or Drop Status

```

1:  $(hdr, payload, crc) \leftarrow \text{ParseFrame}(F)$ 
2: if  $\text{ValidateCRC}(hdr, payload, crc) == \text{FAIL}$  then
3:    $\text{Drop}(F)$ ; return
4: end if
5: if  $hdr.dstAddr \neq myAddr$  and  $hdr.dstAddr \neq 0xFF$  then
6:    $\text{Drop}(F)$ ; return
7: end if
8: if  $hdr.dstAddr == 0xFF$  and  $(hdr.flags \& \text{FLAG\_ACK\_REQ})$  then
9:    $\text{Drop}(F)$ ; return
10: end if
11: if  $hdr.totalChunks == 0$  or  $hdr.chunkIndex \geq hdr.totalChunks$  then
12:    $\text{Drop}(F)$ ; return
13: end if
14: if  $\text{IsCompleted}(hdr.srcAddr, hdr.msgId)$  then
15:   if  $hdr.flags \& \text{FLAG\_ACK\_REQ}$  then
16:      $\text{SendTerminalSack}(hdr.srcAddr, hdr.msgId)$ 
17:   end if
18:    $\text{Drop}(F)$ ; return
19: end if
20:  $session \leftarrow \text{FindSession}(hdr.srcAddr, hdr.msgId)$ 
21: if  $session == \text{NULL}$  then
22:   if  $\text{AvailMem}() < hdr.totalChunks \cdot S_{\max}$  then ▷ Safe stride check
23:      $\text{SendNack}(hdr.srcAddr, \text{BUSY})$ ; return
24:   end if
25:    $session \leftarrow \text{AllocBuf}(hdr.totalChunks, S_{\max})$ 
26: end if
27:  $idx \leftarrow hdr.chunkIndex$ 
28: if  $\text{IsBitSet}(session.bitmap, idx)$  then
29:    $\text{Drop}(F)$  ▷ Duplicate chunk, ignore write
30: else
31:    $\text{WriteChunk}(session.buffer, idx \cdot S_{\max}, payload)$ 
32:    $\text{SetBit}(session.bitmap, idx)$ ;  $session.count \leftarrow session.count + 1$ 
33: end if
34: if  $hdr.flags \& \text{FLAG\_ACK\_REQ}$  then
35:    $\text{SendSack}(hdr.srcAddr, hdr.msgId, session.bitmap)$ 
36: end if
37: if  $session.count == hdr.totalChunks$  then
38:    $P \leftarrow \text{ExtractPayload}(session.buffer)$ 
39:    $\text{DeliverToApp}(P)$ ;  $\text{MarkCompleted}(hdr.srcAddr, hdr.msgId)$ ;  $\text{FreeSession}(session)$ 
40: end if

```

---

#### IV. PRELIMINARY PROOF OF CONCEPT

An initial prototype using commercial Ebyte E220-series modules validated the core segmentation concept. Field trials confirmed feasibility but exposed three limitations: (1) the modules exposed a proprietary network layer that prevented direct control of physical-layer parameters; (2) a large, unoptimized application header inflated per-chunk airtime; and (3) no acknowledgment or retransmission mechanism was available.

These findings motivated the redesign on Semtech SX126x-class transceivers via the RadioLib driver, validated on Heltec Wi-Fi LoRa 32 V3 boards. The revised implementation described in the following sections, is fully hardware-agnostic through the HAL interface and incorporates the 9-byte Logical Header, CRC-16 integrity checks, and the SACK-based retransmission mechanism.

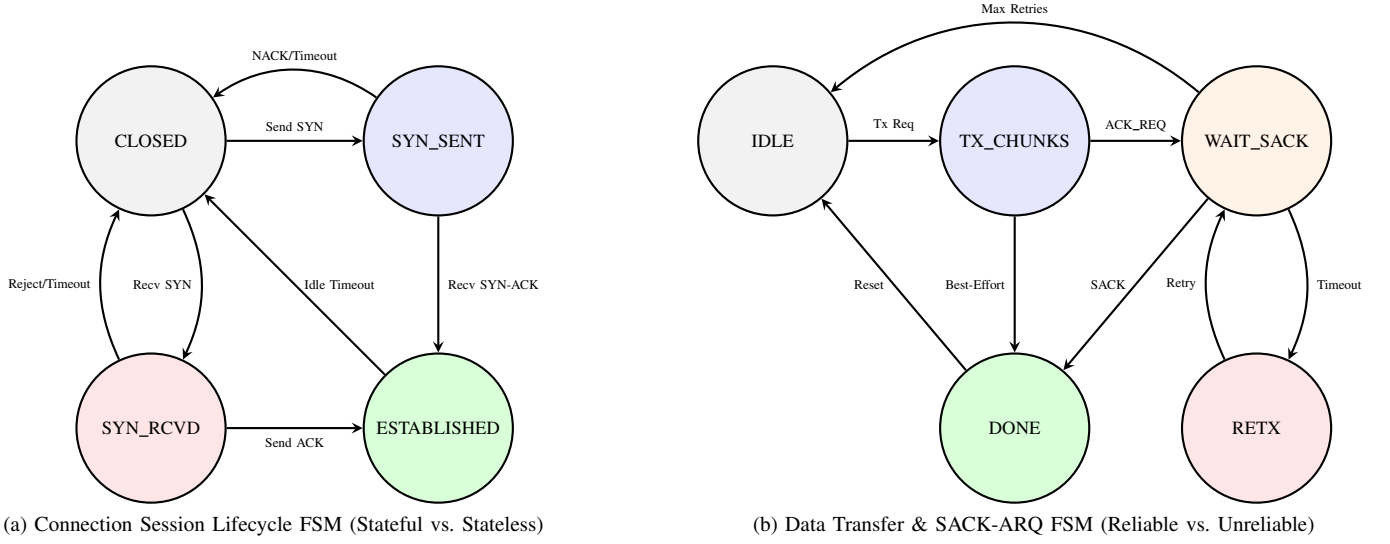


Fig. 2. Finite State Machine (FSM) specifications for connection session lifecycle and data transfer modes.

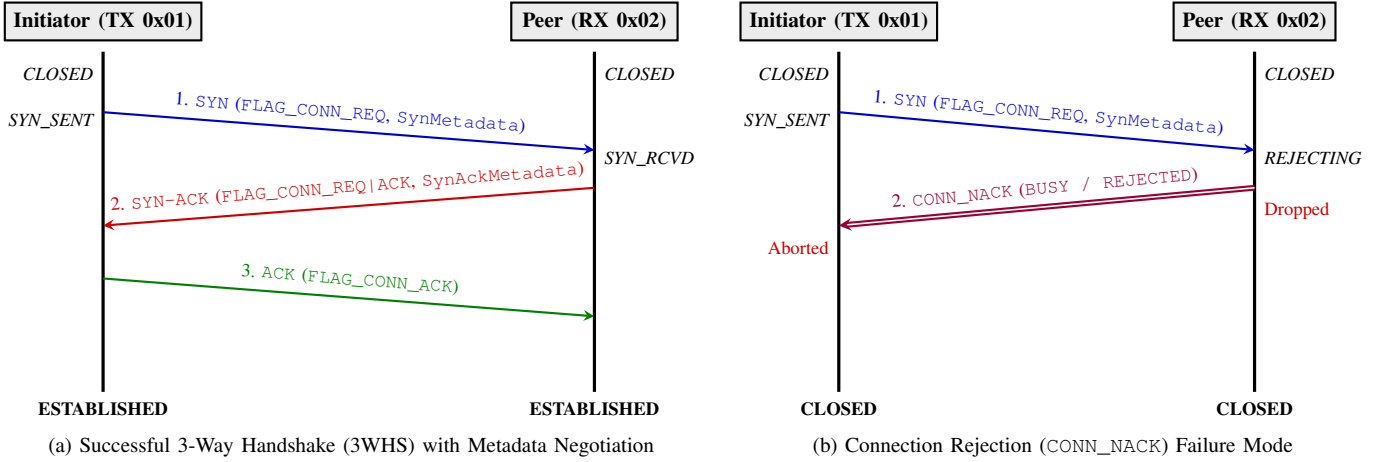


Fig. 3. Sequence interaction flows for 3-Way Handshake (3WHS) connection setup with dynamic metadata negotiation and rejection handling.

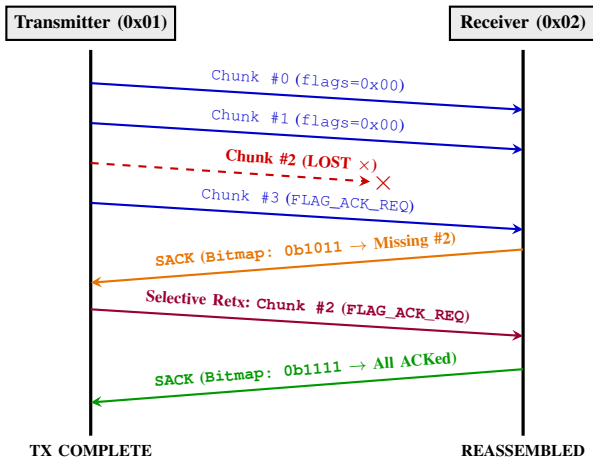


Fig. 4. End-to-end data streaming flow, frame loss detection, and SACK-driven selective retransmission interaction.

## V. HARDWARE LEVEL IMPLEMENTATION

To validate the proposed protocol, a complete implementation of the LAMP library was developed in C++17. The design is modular, separating the transport state machine from physical hardware dependencies and parameter policies.

### A. Centralized Configuration and Parameter Decoupling

To eliminate hardcoded constants and enable deterministic tuning across heterogeneous embedded targets, all protocol operational parameters are centralized within a dedicated compile-time configuration namespace (`LoRaMultiPacketConfig`). This design encapsulates three primary domains:

- Memory and Concurrency Limits:** Governs maximum concurrent reassembly sessions (`MAX_CONCURRENT_SESSIONS = 10`, reducible to 1–2 on memory-constrained Cortex-M0/AVR devices), physical FIFO buffer bounds (`PHY_BUFFER_SIZE = 256` bytes, reflecting transceiver hardware FIFO capacity)



supporting the 255-byte maximum LoRa PHY payload cap), and completed sequence deduplication depth (`MAX_COMPLETED_HISTORY` = 16).

- **Timing and Retransmission Policies:** Configures the default retry ceiling ( $R_{\max} = 5$ ), minimum SACK timeout floor ( $T_{\min} = 1000$  ms, tunable for low-latency P2P links), safety multiplier ( $\alpha = 1.5$ ), and half-duplex switching guard ( $\delta = 25$  ms).
- **Session Lifecycle Thresholds:** Defines handshake initiation timeouts (`DEFAULT_SYN_TIMEOUT_MS` = 3000 ms), connection inactivity teardown periods (`DEFAULT_CONN_INACTIVITY_TIMEOUT_MS` = 15000 ms), and stale session pruning timers (`PRUNE_TIMEOUT_MS` = 15000 ms).

Complementing the compile-time configuration, the 3WHS mechanism enables dynamic run-time parameter negotiation (`SynMetadata`), providing a two-tier flexibility model that adapts to variable channel conditions and peer memory capabilities.

### B. Hardware Abstraction Layer (HAL)

To keep the protocol core hardware-agnostic, a clean HAL interface (`RadioLibHal`) was implemented. On the physical nodes, the concrete class `EspHal` wraps ESP32-S3 hardware resources: Serial Peripheral Interface (SPI) Communication (register configuration and FIFO buffer transfers for the Semtech SX1262), General-Purpose Input/Output (GPIO) Interrupts (NSS, RST, BUSY, and the DIO1 RX/TX completion interrupt pin), and Timing Primitives (`millis()` and `delay()`).

### C. Host-Based Unit Testing

Following a TDD methodology, the protocol core was validated in isolation on a desktop host using the **Unity** framework in PlatformIO, verifying: (i) Modbus CRC-16 calculation on headers and partial/padded payloads, (ii) reassembler state transitions under out-of-order and duplicate chunk arrivals, and (iii) deterministic pruning of stale session descriptors.

### D. Physical Benchmarking Setup

Physical validation was conducted using two Heltec Wi-Fi LoRa 32 V3 development boards (ESP32-S3 + Semtech SX1262) configured with: Carrier Frequency 869.525 MHz, SF7, BW 500.0 kHz, CR 4/5, TX Power +10 dBm, and 8-symbol preamble.

1) *Regulatory Compliance and Regional Deployment:* It is important to clarify the relationship between the LAMP transport protocol and RF regulatory constraints. LAMP operates strictly at the *transport layer* and is agnostic to regional carrier frequency, channel width, and RF power limits; compliance with these constraints is the responsibility of the underlying MAC/PHY layer.

Under European ETSI EN 300 220-2 regulations [6], the 869.400–869.650 MHz sub-band (Band P) allocates a 250 kHz total frequency block permitting up to 10% duty cycle and an

output power up to 500 mW ERP (27 dBm). For field deployments strictly adhering to this sub-band, channel bandwidths of  $\leq 250$  kHz (specifically 125 kHz or 250 kHz raster centered at 869.525 MHz) are selected. To characterize the protocol's upper theoretical throughput limits under maximum channel stress, our simulation campaign and laboratory benchmarks evaluated a wideband configuration ( $BW = 500.0$  kHz, matching standard US915 / AU915 wideband LoRa allocations) on an experimental carrier.

Additionally, the reference C++ library provides opt-in MAC-layer helpers, including SX1262-native Channel Activity Detection (CAD) with randomized exponential backoff (CSMA-CA) and duty-cycle pacing to mitigate channel contention. These features are decoupled from the transport core and offered as convenience primitives for integrators targeting shared ISM spectrum.

## VI. PERFORMANCE EVALUATION AND EXPERIMENTAL METHODOLOGY

To evaluate the reliability, throughput, and communication limits of LAMP, we adopt a two-stage evaluation methodology: a large-scale stochastic network simulation campaign executed in ns-3, complemented by functional hardware testbed validation on microcontroller nodes and a planned outdoor field benchmarking campaign.

### A. Network Simulation Benchmarking (ns-3)

To characterize protocol performance under controlled fading channels across extensive distance ranges, we developed a full C++ simulator integrated into the ns-3 framework (`ns3-lorawan`). The simulator reproduces the exact LAMPv2 frame structure (9-byte Logical Header + 2-byte CRC-16), all four operational modes, the 3WHS handshake, and selective bit-vector SACK retransmissions over a Nakagami- $m$  fast-fading propagation loss model calibrated for the Semtech SX1262 transceiver (+24.2 dBm EIRP, 869.525 MHz).

1) *Simulation Campaign Hierarchy:* To achieve statistical convergence, an extensive Monte Carlo campaign comprising 117,050 total independent simulation runs was conducted. The experimental space spans 3 RF propagation environments (LoS, Urban, Rural), 3 Spreading Factors (SF7, SF9, SF10), 4 operational modes (Mode 1 to Mode 4), and 50 independent random seeds per parameter configuration across an adaptive distance grid extending dynamically up to 23.5 km with early-stopping thresholds once reception falls to 0% across consecutive steps. Each curve point plotted in Figs. 5 and 6 represents the sample mean over 50 independent Monte Carlo seed replications for a representative  $N = 20$ -chunk payload (4.88 KB data size), with shaded ribbons denoting 95% confidence intervals.

2) *Analytical Reliability Bound under Stated Assumptions:* To model payload completion probabilities analytically, we define a theoretical baseline under stated hypotheses:

- 1) *Independent Loss Model:* Channel frame loss is modeled as an independent and identically distributed (i.i.d.) Bernoulli error process with packet error rate  $p \in [0, 1]$

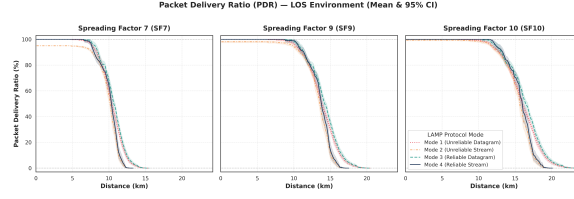


Fig. 5. Packet Delivery Ratio ( $PDR_{app}$  %) for a fixed  $N = 20$ -chunk payload transfer (4.88 KB application data) as a function of distance across Spreading Factors (SF7, SF9, SF10) in Line-of-Sight (LoS) propagation. Solid lines denote mean delivery rates across 50 independent Monte Carlo random seeds; shaded ribbons represent 95% confidence intervals.

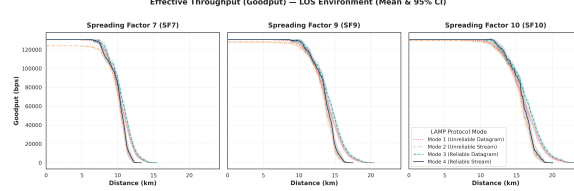


Fig. 6. Effective application-layer throughput (Goodput in bps) for an  $N = 20$ -chunk payload transfer across Spreading Factors and LAMP operational modes in LoS. Shaded ribbons denote 95% confidence intervals across 50 Monte Carlo seeds.

and error-free control feedback ( $p_{SACK} = 0$ ). While the i.i.d. Bernoulli model provides a tractable analytical baseline, physical LPWAN channels typically exhibit temporally correlated fading.

- 2) *Bounded Repair Rounds*: Selective SACK retransmissions are bounded by the protocol's maximum repair round budget  $R_{max} = 5$  (matching `LoRaMultiPacketConfig::MAX_RETRIES`).

Under these idealized assumptions, the analytical upper bound for successfully reassembling an  $N$ -fragment payload across  $R_{max}$  retries matches Eq. (4):

$$P_{complete}(N, p, R_{max}) = (1 - p^{R_{max}+1})^N \quad (6)$$

Because Algorithm 1 enforces a global session repair round budget ( $R_{max} = 5$ ) across all missing fragments rather than independent per-chunk retries, Eq. (6) serves as an optimistic theoretical baseline. Empirical multi-chunk delivery dynamics under correlated channel fading and joint forward-reverse link losses are directly evaluated via the stochastic ns-3 campaign. It is crucial to clarify that when reliable modes (Mode 3 and Mode 4) sustain a 100% Application Payload Delivery Ratio ( $PDR_{app}$ ) in Fig. 5, this does not indicate physical channel reliability or zero-loss propagation. Rather, it demonstrates that the selective bit-vector SACK-ARQ mechanism successfully recovers lost fragments within the  $R_{max} = 5$  retry budget prior to session timeout.

In LoS conditions, SF10 sustains complete payload reassembly up to 19.1 km, exhibiting a steep drop-off between 20.0 km and 23.1 km as frame loss probability exceeds retransmission recovery budgets. Conversely, best-effort unreliable modes (Mode 1 and Mode 2) experience progressive packet drop rates as spatial fading degrades signal-to-noise ratios.

Fig. 6 depicts effective application-layer Goodput (bps). While best-effort streaming (Mode 2) achieves peak throughput in pristine channels by omitting acknowledgment latency, Mode 4 (*Reliable Stream*) preserves sustained throughput near physical channel capacity across the operational range, degrading only near receiver sensitivity thresholds.

## B. Planned Real-World Field Benchmarking Campaign

While hardware testbed experiments on two Heltec Wi-Fi LoRa 32 V3 nodes have validated the functional correctness, C++ memory safety, 3WHS state machine transitions, and out-of-order reassembly of LAMP, a systematic outdoor field campaign is planned as future experimental work to record real-world empirical performance under environmental interference, multipath reflections, and temperature-induced frequency drift.

### 1) Planned Experimental Matrix and Evaluation Metrics:

The planned outdoor physical validation strategy follows established LPWAN characterization methodologies [10]. It will sweep transmission distances (1 to 5000 meters across LoS and Non-Line-of-Sight (NLoS) conditions) and payload sequence sizes (1 to 100 chunks,  $\approx 24.6$  KB) under standard radio configurations (SF7, BW 500 kHz, CR 4/5, 10 dBm TX power).

During the planned outdoor campaign, four core performance metrics will be recorded across 5 independent trials per operational mode:

- **Application Payload Delivery Ratio ( $PDR_{app}$ )**: The percentage of application-level payloads successfully reconstructed at the receiver.
- **End-to-End Latency**: Total elapsed duration from initial fragment serialization at the transmitter to final reassembly callback at the receiver.
- **Effective Throughput (Goodput)**: The delivery rate of application payload data, calculated as  $\text{PayloadSize (bits)} / \text{Latency (seconds)}$ .
- **Retransmission Overhead**: Count of extra fragments and SACK frames transmitted in reliable modes compared to unreliable baselines.

## VII. CONCLUSION AND ARCHITECTURAL ROADMAP

This paper presents the design, implementation, network simulation, and hardware validation of LAMP, a lightweight and reliable transport-layer architecture optimized for transmitting segmented large payloads over point-to-point and local

multi-node LoRa networks. By replacing traditional Stop-and-Wait per-packet acknowledgments with selective retransmissions requested via cumulative binary bitmaps, and eliminating redundant SOM/EOM markers in favor of implicit index derivation, the protocol achieves high reliability under lossy conditions with minimal airtime overhead and duty-cycle footprint.

#### A. Protocol Scope and Limitations

To provide a transparent systems assessment, three inherent architectural boundaries of baseline LAMPv2 are identified:

- 1) *Session Capacity Boundary*: A single transport session is bounded to  $255 \text{ chunks} \times 244 \text{ B} = 62,220 \text{ bytes}$  ( $\approx 62.2 \text{ KB}$ ). Larger files require application-layer multi-session chunking.
- 2) *Pessimistic Buffer Pre-allocation*: To guarantee memory safety against out-of-order partial tail chunks arriving first, the receiver allocates contiguous memory based on  $N \times S_{\max}$  upfront. On RAM-constrained micro-controllers ( $\leq 32 \text{ KB SRAM}$ ), the concurrency pool (`MAX_CONCURRENT_SESSIONS`) must be set to 1–2 sessions.
- 3) *Unauthenticated Baseline 3WHS*: Baseline LAMPv2 incorporates CRC-16 for physical frame integrity but omits cryptographic nonces. Consequently, adversarial replays of stale SYN frames could exhaust active session descriptors. This is mitigated via the opt-in Level-2 Security Profile (AES-128-GCM / HMAC) detailed below.

#### B. Architectural Roadmap

To guide the long-term progression of LAMP from point-to-point data links to decentralized multiagent mesh networks, we define a three-level architectural roadmap:

- **Level 1 – Addressed Transport & Polite Spectrum Access (Current Stack)**: 8-bit addressing (`srcAddr`, `dstAddr`), 9-byte header, four operational modes, dynamic 3WHS, and opt-in CAD/CSMA-CA under ETSI EN 300 220-1 [6].
- **Level 2 – Adaptive Spectrum, Scheduling & Security (Near-Term)**: Dynamic ADR tuning via RSSI/SNR metrics in SACK feedback, time-slotted channel access (TDM) [15], and an optional transport security profile (AES-128-GCM / HMAC-SHA256 with 32-bit replay counters).
- **Level 3 – Multi-Hop Mesh & IPv6 Extension (Long-Term Vision)**: Dynamic multi-hop routing (AODV-Light / RPL adaptation) with hop-by-hop vs. end-to-end SACK reliability [13], [14], and 8-bit node ID mapping for IPv6 link-local bridging (`fe80::/64`).

#### DECLARATIONS

**Data and Code Availability**: The complete C++17 library source code, HAL drivers, Unity test suites, and ns-3 simulation scripts are publicly available on GitHub at <https://github.com/aleemont/LoRaMultiPacket> under the MIT license.

**Ethical Statement & AI Tool Usage**: This study did not involve human or animal subjects. AI assistant tools were utilized strictly for code formatting, micro-typography auditing, and manuscript style calibration in compliance with IEEE publication ethics guidelines.

**Conflict of Interest**: The authors declare that they have no competing financial or personal interests that could influence the work reported in this paper.

#### REFERENCES

- [1] M. Radeta, J. Pestana, P. Abreu, R. Freitas, F. Silva, D. Vieira, R. Prieto, M. Fernandez, F. Alves, T. Dellinger, S. Neves, and E. Delory, “Triton—open telemetry and location estimation for marine monitoring based on iot and lora,” *IEEE Journal of Oceanic Engineering*, vol. 50, no. 2, pp. 1244–1258, 2025.
- [2] T. Elshabrawy and J. Al-Ali, “Lora technology demystified: From link behavior to cell capacity,” *IEEE Transactions on Wireless Communications*, vol. 17, no. 10, pp. 6611–6621, 2018.
- [3] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, “Understanding the limits of lorawan implementations,” *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.
- [4] S. De, H. M. Jalajamony, S. Adhinarayanan, S. Joshi, H. Upadhyay, and R. Fernandez, “Multimedia transmission over lora networks for iot applications: A survey of strategies, deployments, and open challenges,” *Sensors*, vol. 25, no. 23, p. 7128, 2025.
- [5] P. J. Marcelis, V. S. Rao, and R. V. Prasad, “Dare: Data recovery through application layer coding for lorawan,” in *Proceedings of the 2017 IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2017.
- [6] *Short Range Devices (SRD) operating in the frequency range 25 MHz to 1 000 MHz; Part 1: Technical characteristics and methods of measurement*, European Telecommunications Standards Institute (ETSI) Std. ETSI EN 300 220-1 V3.1.1, 2017.
- [7] A. Minaburo, L. Toutain, C. Gomez, D. Barthel, and J. C. Zúñiga, “SCHC: Static context header compression and fragmentation for LP-WAN,” Internet Engineering Task Force (IETF), RFC 8724, April 2020.
- [8] C. Gomez and J. Crowcroft, “Static context header compression (SCHC) over LoRaWAN,” Internet Engineering Task Force (IETF), RFC 9011, April 2021.
- [9] LoRa Alliance Technical Committee, “LoRaWAN fragmented data block transport specification,” LoRa Alliance, Technical Specification TS004-2.0.0, 2022.
- [10] M. Cattani, C. A. Boano, and K. Römer, “An experimental evaluation of the reliability of lora long-range low-power wireless communication,” *Journal of Sensor and Actuator Networks*, vol. 6, no. 2, p. 7, 2017.
- [11] M. Bor, U. Roedig, T. Voigt, and J. M. Alonso, “Do lora low-power wide-area networks scale?” in *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2016, pp. 59–66.
- [12] T. Chen, D. Eager, and D. Makaroff, “Efficient image transmission using lora technology in agricultural monitoring iot systems,” in *2019 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2019, pp. 937–944.
- [13] J. Kim, J. Jenkins, J. Seol, and M. Kwak, “Extending data transmission in the multi-hop lora network,” *Issues in Information Systems*, vol. 23, no. 3, pp. 292–304, 2022.
- [14] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, “Multi-hop and mesh for lora networks: Recent advancements, issues, and recommended applications,” *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–43, 2024.
- [15] C.-C. Wei, J.-K. Huang, C.-C. Chang, and K.-C. Chang, “The development of lora image transmission based on time division multiplexing,” in *2020 International Symposium on Computer, Consumer and Control (IS3C)*, 2020, pp. 323–326.
- [16] A. Calabrò, E. Marchetti, and M. T. Paratore, “Evaluating use of arq strategies in communication protocols for search and rescue,” in *Proceedings of the 21st International Conference on Web Information Systems and Technologies (WEBIST)*. SCITEPRESS, 2025, pp. 59–70.
- [17] S. Corporation, “Lora modem designer’s guide,” Semtech Corporation, Tech. Rep. Application Note AN1200.13, 2015.